Euclid's Game Boy: CasNum, a Compass-and-Straightedge Arbitrary-Precision Arithmetic Library

Omer Goldzweig (0x0mer)

March 2026

Abstract

We present CasNum [8], a Python library for arbitrary-precision arithmetic built entirely from compass-and-straightedge constructions. Numbers are represented as points on the x-axis, arithmetic is reconstructed from classical geometry, and the resulting system is expressive enough to support increasingly questionable applications, including RSA and the ALU of a Game Boy emulator. We describe the theoretical model, the implementation compromises, and the practical costs of recovering arithmetic from classical geometry.

1 Introduction

The contemporary programmer enjoys an undeserved level of arithmetic convenience. Expressions such as $a + b$ are evaluated by dedicated hardware, often in a single instruction. CasNum explores a less convenient but more humble approach: representing numbers as geometric objects and implementing arithmetic through classical constructions. Under this model, addition is no longer a primitive operation but a theorem with runtime implications. The practical disadvantages of this approach are substantial, yet its potential moral advantages are hard to ignore.

This is not a new idea in geometry. Euclid already gave us the primitive operations in *Elements* [1]. What is new is the insistence that this simple geometric toolkit should now carry modular arithmetic, bitwise logic, toy RSA [4], and part of a Game Boy emulator [2]. Few papers, if any, have managed to cite Euclid, RSA and *Pokémon Red Version* [5] while remaining, in some narrow technical sense, a coherent document. We regard this as one of the project's stronger results.

It is worth clarifying in what sense CasNum is "arbitrary precision." The library represents all numbers exactly, using symbolic expressions rather than floating-point approximations, and performs all derived arithmetic at that symbolic level. The only approximation enters when the implementation must distinguish between multiple candidate roots, in which case it uses numerical comparison at a user-chosen precision. Accordingly, the system is most accurately described as an exact symbolic mathematics library whose correctness is limited only by the precision used for those comparisons.

2 Model

2.1 Compass and Straightedge Constructions

In compass-and-straightedge geometry, one begins with only two points: the origin $O = (0, 0)$ and the unit $U = (1, 0)$, and with the following (mercifully small) API:

1. Given two distinct points, construct the unique line through them

2. Given two distinct points, construct the unique circle centered at one and passing through the other
3. Intersect two non-parallel lines
4. Intersect a line and a circle
5. Intersect two circles

From here, with sufficient patience, everything else is derived. For example, given two points A and B, the midpoint of the segment $\overline{AB}$ is obtained as follows: construct the circles centered at A and B through B and A, respectively. Then, take their two intersection points, and pass a line through them. This resulting line is the perpendicular bisector of $\overline{AB}$, and thus intersecting it with $\overline{AB}$ we obtain the desired midpoint.

CasNum implements the above as a small geometry engine, which serves as its basis. Coordinates are stored as symbolic SymPy expressions rather than floating-point approximations [3], which preserves exactness while causing the resulting radical expressions to rapidly deteriorate in readability.

2.2 Number Representation

While in the classic theory of compass-and-straightedge constructions, a number r is said to be constructible if one can create a segment of length r, in CasNum we instead represent a number x as the point $(x, 0)$ on the x-axis.

The project includes two ways to construct integers. One is through repeated unit extension (that is, repeatedly adding the unit to itself to obtain $2, 3, 4, \ldots$), which is mathematically pure but slow. The other decomposes the input in binary and rebuilds it by doubling and addition. The source code explicitly asks whether the latter counts as cheating. We regard this as a strength – a system that cannot name its own compromise is not ready for publication.

2.3 Comparison and Control Flow

Number comparison is done by comparing x-coordinates. This quietly grants the system one extra primitive: deciding which of two points on the x-axis lies farther to the right. Such an order test is not normally listed among the formal compass-and-straightedge operations, but it seems reasonable to imagine that a sufficiently well-sighted practitioner would be able to tell which point lies farther to the right, once the points have been constructed. In our implementation, that role is played by SymPy rather than by the naked eye. We further allow the outcome of such comparisons to determine the next step of the construction process, including through branching and loops. This does not change which numbers can appear as outputs of a terminating computation, since every such computation still consists of a finite sequence of ordinary constructions. It does, however, enlarge the class of algorithms we allow ourselves to run in order to obtain them.

3 Arithmetic by Construction

In this section, we shall describe how each construction is done. As these can be nice riddles, the spoiler-avoiding reader may freely skip ahead to Section 4. In the pseudocode we shall refer to the origin as 0 and to the unit as 1. Once we build addition, we will also freely appeal to 2 as a constant that we know how to build and use.

3.1 Negation

Negation is the cleanest operation in the system: given a point $P = (p, 0)$, construct the circle centered at the origin through P and take its second intersection with the x-axis. The result is $(-p, 0)$. Negation is therefore implemented as literal reflection through the origin, which is a pleasant way of saying "invert the sign." With negation in place, and given the primitive of comparing numbers, we can now take absolute value of a number, which shall be denoted by $|\cdot|$ as usual.

3.2 Doubling and Halving

Doubling is implemented in a similar fashion: Given a point $P = (p, 0)$, the implementation now constructs the circle centered at P through the origin and takes its second intersection with the x-axis. That point is $(2p, 0)$, because both radii have length p. In the source this appears as `double_point_on_x_axis(origin, p)`. Halving is just the midpoint of the segment from the origin to P. Since the general midpoint construction was already described above, the implementation here is simply that routine applied to $\overline{OP}$, yielding $H = (p/2, 0)$ via `half_point_on_x_axis(origin, p)`.

3.3 Addition and Subtraction

With doubling available, the classical midpoint construction for addition is immediate. Given points $A = (a, 0)$ and $B = (b, 0)$, construct the midpoint M of the segment $\overline{AB}$. Since $M = ((a + b)/2, 0)$, doubling M from the origin yields $(a + b, 0)$. Subtraction is then addition after mirroring one operand through the origin.

CasNum also provides a faster addition path. If a previously constructed radius may be transferred to a new center, the library draws the circle centered at a with radius $|b|$ and intersects it with the x-axis. This is the "non-collapsing-compass" assumption. It does not add mathematical power, but it does save construction work, which in software is the form of power that matters.

In this codebase, the fast route avoids the perpendicular bisector of $\overline{AB}$, the two auxiliary circles behind that bisector, and the extra circle used for doubling. The library therefore defaults to the faster path. Purists may disable it by changing the relevant flag and accepting the corresponding loss in throughput. Algorithm 1 below describes the algorithm in pseudocode.

Algorithm 1 GeometricAdd(a, b)

Require: CasNum values a, b represented on the x-axis by the points A, B, respectively.
if $a = 0$ **then return** copy of b
else if $b = 0$ **then return** copy of a
else if $a = b$ **then**
    return Mul2(a)
end if
if radius transfer is allowed **then**
    Construct circle C centered at a with radius $|b|$
    Let $\{p_1, p_2\} \leftarrow C \cap x$-axis
    if $b > 0$ **then**
        return rightmost point among $\{p_1, p_2\}$
    else
        return leftmost point among $\{p_1, p_2\}$
    end if
else
    Let m be the midpoint of segment $\overline{AB}$
    return Mul2(m)
end if

3.4 Additional Constructions

Additional standard compass-and-straightedge constructions that we shall use in the following sections, and that are worth mentioning are:

- Constructing the unique line parallel to a given line through a given point.
- Constructing the unique line perpendicular to a given line through a given point.

The details of these constructions are left as an exercise to the reader.

3.5 Multiplication and Division

The multiplication, division, and square-root constructions are classical textbook geometry rather than novel contributions of this work. In practice, the implementation was guided in particular by Root Beer's expository videos on constructible multiplication and square roots [6, 7], which were helpful in translating the classical diagrams into code.

Multiplication and division are implemented through triangle similarity. The pseudocode is given below.

Algorithm 2 GeometricMul(a, b) and GeometricDiv(a, b)

Require: CasNum values a, b

if division and $b = 0$ **then**
    Error on division by 0
else if $a = 0$ or (multiplication and $b = 0$) **then**
    return 0
end if
Lift $(|a|, 0)$ to $P = (0, |a|)$ using a circle centered at the origin
if multiplication **then**
    Let L be the line through P and $(-1, 0)$
    Draw the line through $(|b|, 0)$ parallel to L
    Let Q be its intersection with the y-axis. Then we have $Q = (0, -|a||b|)$
else
    Let L be the line through P and $(|b|, 0)$
    Draw the line through $(-1, 0)$ parallel to L
    Let Q be its intersection with the y-axis. Then we have $Q = (0, -|a|/|b|)$
end if
Project the radius $|OQ|$ back to the positive x-axis using a circle centered at the origin; call the result r
if exactly one of a, b is negative **then**
    $r \leftarrow -r$
end if
return r

Figure 1 below shows the positive multiplication construction used in the implementation.

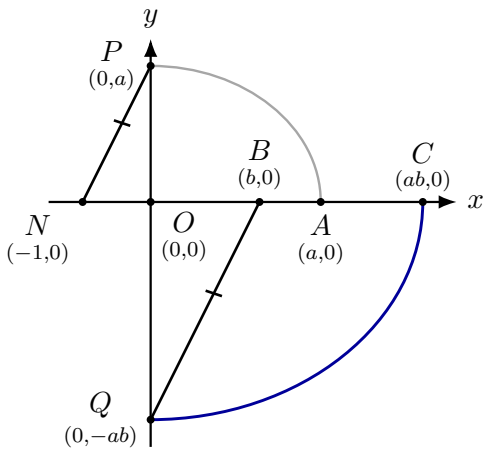

Figure 1: Positive multiplication. The gray arc copies a to the y-axis. Negating the unit allows us to build the line NP. Then, a helper routine that builds a parallel line through a given point allows us to build BQ as the parallel line to NP through B. $BQ \parallel NP$ makes the right triangles $\triangle ONP$ and $\triangle OBQ$ similar, and finally the blue arc returns the radius $|OQ| = ab$ to the x-axis.

3.6 Square Root

Square root uses a classical construction based on the identity

$$\left(\frac{x+1}{2}\right)^2 - \left(\frac{x-1}{2}\right)^2 = x.$$

Figure 2 shows the geometry used by `CasNum.sqrt()`.

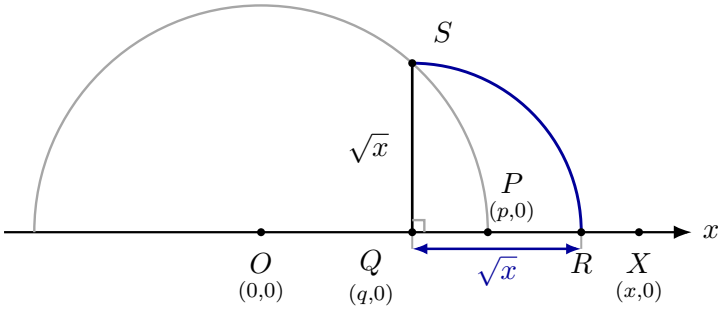

Figure 2: Square root construction. Let $p = (x+1)/2$ and $q = (x-1)/2$. The point S lies on the circle centered at the origin with radius p and on the vertical through Q, so using Pythagorean theorem for the triangle OQS (where $|OQ| = q, |OS| = p$), we get $QS^2 = p^2 - q^2 = x$. The blue arc is a circle centered at Q with radius QS, placing the same length on the axis at R. Subtracting Q from R yields $\sqrt{x}$.

3.7 Modulo, Shifts, and Bitwise Logic

The arithmetic becomes more interesting when it stops being naturally geometric. This section could theoretically be a genuinely new theorem that CasNum introduces, not because it is in any way non-trivial, but probably due to lack of basic interest or possibly coherence. All the operations in this section are only well defined for integers. Modulo is implemented by repeatedly subtracting

the largest multiple of the divisor that does not overshoot the dividend. Right shift halves even values and adjusts odd values first, thereby matching integer floor semantics. Left shift is repeated doubling.

Bitwise `AND`, `OR`, and `XOR` are then built on top of modulo-by-two, shifting, and reconstruction from powers of two. This is not elegant in the algebraic sense, but algebraic elegance was not what we set out to accomplish. The pseudocode for `XOR` is given in Algorithm 3. The other bitwise operations are implemented similarly.

Algorithm 3 GeometricXorPositive(a, b)

Require: non-negative integer CasNum values a, b

$result \leftarrow 0$
$power \leftarrow 1$
$two \leftarrow 2$
while $a > 0$ or $b > 0$ **do**
    $bit_a \leftarrow a \bmod two$
    $bit_b \leftarrow b \bmod two$
    if $bit_a + bit_b = 1$ **then**
        $result \leftarrow result + power$
    end if
    $a \leftarrow a \gg 1$
    $b \leftarrow b \gg 1$
    $power \leftarrow 2 \cdot power$
end while
return $result$

The signed case is handled by first translating any negative input into a sufficiently wide two's-complement window, running Algorithm 3, and then translating the result back.

4 From Arithmetic Stack to Euclidean ALU

Arithmetic alone is merely a rather useless library. To obtain a rather useless systems result, we integrated CasNum into the ALU of PyBoy [2], a Python-based Game Boy emulator, thereby producing what is, to the best of our knowledge, the world's first fully functional compass-and-straightedge ALU. After integration, every ALU opcode is implemented entirely using CasNum. This was achieved by editing PyBoy's opcode generator: the generated handlers first lift ordinary 8-bit or 16-bit operands into CasNum values via `CasNum.get_n`, perform the requested operation geometrically, and then write the resulting integer back into the register by extracting its x-coordinate. The flag register is updated accordingly, using CasNum, of course.

For example, prior to our modifications, the generated handler for `ADC A,B` was the standard sort of emulator routine, still burdened with native arithmetic. Roughly:

```
a = cpu.A
b = cpu.B
c = ((cpu.F & (1 << FLAGC)) != 0)
t = a + b + c
flag |= ((t & 0xFF) == 0) << FLAGZ
flag |= (((a & 0xF) + (b & 0xF) + c) > 0xF) << FLAGH
flag |= (t > 0xFF) << FLAGC
cpu.A = t & 0xFF
```

After Euclideanization, the same handler becomes:

```
a = CasNum.get_n(cpu.A)
b = CasNum.get_n(cpu.B)
c = CasNum.get_n((CasNum.get_n(cpu.F)
                  .get_nth_bit(FLAGC)) != zero)
t = a + b + c # The ‘+‘ operator is overloaded in the CasNum class
flag |= (CasNum.get_n((t & CasNum.get_n(0xFF)) == zero) << FLAGZ)
flag |= (CasNum.get_n(((a & CasNum.get_n(0xF)) + (b & CasNum.get_n(0xF))
                       + CasNum.get_n((CasNum.get_n(cpu.F).get_nth_bit(FLAGC)) != zero))
                      > CasNum.get_n(0xF)) << FLAGH)
flag |= (CasNum.get_n(t > CasNum.get_n(0xFF)) << FLAGC)
cpu.A = int((t & CasNum.get_n(0xFF)).p.x)
```

The notable fact is that the opcode's control skeleton survives intact: operands are loaded, carry is added, flags are computed, the result is masked to 8 bits, and the value is written back to the register. The intervention is simply that every arithmetic operation is now forced through CasNum, and only the final answer is projected back down to an ordinary integer.

The first boot of Pokémon Red [5] takes on the order of fifteen minutes, which is unheard of in the bad sense for an emulator and unheard of in the good sense for a straightedge.

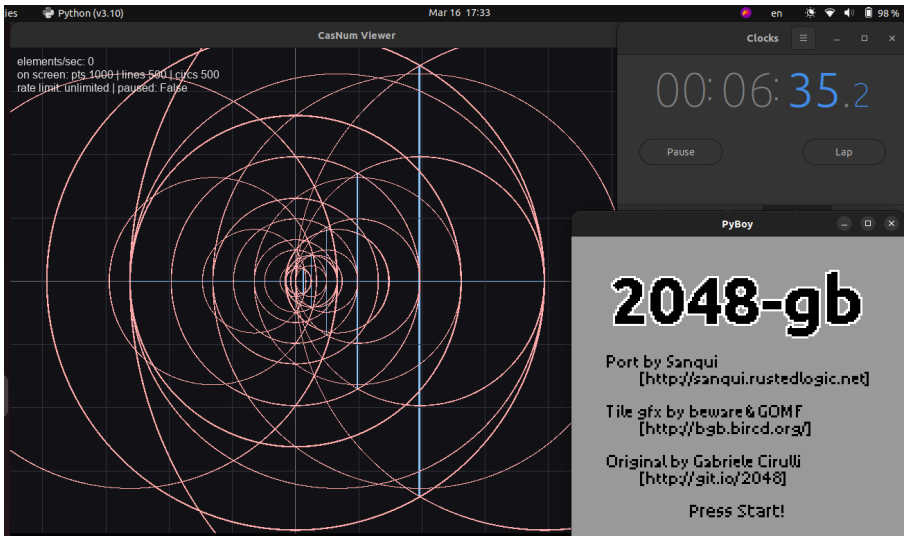

Figure 3: PyBoy running under the CasNum ALU. To the left of the emulator display is a visualization of the underlying geometric constructions. The stopwatch in the upper-right shows that reaching the start screen for 2048 took approximately 6 and a half minutes. We depict 2048 rather than Pokémon Red out of respect for copyright law and, more pressingly, SIGBOVIK's explicit warning that DMCA notices are a hassle.

5 Notes on Purity, Performance, and Usefulness

CasNum mixes classical constructions with a few deliberate conveniences. Integer construction is accelerated by binary decomposition. Fast addition assumes radius transfer. Caching lets the system reuse earlier constructions instead of reconstructing the same lines, circles, and intersections each time. These are compromises we are willing to make, as a perfectly pure implementation would mainly differ by being slower.

SymPy [3] is used throughout the code to simplify the monstrous closed-form radicals produced by the constructions. However, comparing such expressions directly is not always straightforward. When the implementation must choose between multiple candidate solutions, as in quadratic constructions, it falls back on high-precision numerical comparison to determine the intended one. A possible remedy for this impurity is discussed in Section 6.

As for complexity, the project README gives the time complexity as "Yes" and the space complexity as "Also yes." This is difficult to improve upon. Nearly every operation is cached with `lru_cache`, allowing the system to participate in the time-honored tradition of trading time for space.

We found that implementing these operations from scratch leaves considerable room for creative optimization. For example, the modulo routine is accelerated by repeatedly subtracting the greatest power-of-two multiple of the divisor that does not exceed the dividend, yielding an exponential improvement in runtime over naive repeated subtraction.

We were asked several times whether any operations turn out to be unusually efficient in this architecture. The closest example we found was square root: at the level of geometric constructions, it requires only a constant number of steps to produce an exact result, in contrast to the iterative approximation procedures more common in conventional arithmetic implementations. In practice, however, our implementation still incurs the cost of symbolic manipulation, which is somewhat harder to reason about. Nevertheless, from a purely theoretical perspective, square root is a constant-time exact operation. We would be interested to hear of other operations that benefit similarly from the model.

6 Future work

The next step practically suggests itself: having laid the groundwork, we are finally ready to implement CasCasNum, a library for arbitrary-precision arithmetic using nothing more than compass-and-straightedge constructions and CasNum integers. This would introduce a second layer of geometric virtualization and, more importantly, make the current version look restrained.

In addition, a more complete implementation of "Euclid's Game Boy" would represent not only arithmetic but the entire processor state geometrically. For example, RAM could be encoded by a distinguished ray, say $y = x$, while the ith byte is represented by a ray from $(i, 0)$. The x-coordinate of the intersection between that byte-ray and the RAM-ray would then determine the stored value of the ith byte in memory. This scheme is admittedly more forced than the arithmetic layer, motivated primarily by a desire to leave no component unemancipated from conventional computation. The RAM is only one example – similar encodings could presumably be devised for most other relevant components of the emulator as well, perhaps with the exception of audio, whose geometric destiny remains less clear. A possible direction that we do find as natural as the Game Boy ALU is implementing the Game Boy's graphics geometrically as well. Instead of drawing pixels in memory and displaying them on a screen, one could imagine rendering each frame directly through circles and lines on the board. This would make the visual output consistent with the arithmetic that produced it.

As noted in the previous section, the library currently relies on approximations to distinguish between candidate points. This impurity can be removed by attaching to each number an isolating segment containing the intended root while excluding the other roots of its minimal polynomial. At that point, the system would finally be exact all the way down.

7 Conclusion

CasNum is a project that attempts to answer a question: how far can geometric exactness be carried into software before the costs render it completely useless? Far enough, apparently, to support RSA [4] and to make measurable progress on Pokémon, provided one is not in a hurry and is willing to accept a sufficiently weak definition of "interactive system."

One still cannot square the circle using compass-and-straightedge. One can, however, use them to help run Pokémon Red. Surely the ancient Greeks would have loved to see this.

References

[1] Euclid. *Elements*. Alexandria, circa 300 BCE.

[2] M. Y. Baekalfen. PyBoy: Game Boy emulator written in Python. https://github.com/Baekalfen/PyBoy

[3] Meurer, Aaron and Smith, Christopher P. and Paprocki, Mateusz and Čertík, Ondřej and Kirpichev, Sergey B. and Rocklin, Matthew and Kumar, AMiT and Ivanov, Sergiu and Moore, Jason K. and Singh, Sartaj and Rathnayake, Thilina and Vig, Sean and Granger, Brian E. and Muller, Richard P. and Bonazzi, Francesco and Gupta, Harsh and Vats, Shivam and Johansson, Fredrik and Pedregosa, Fabian and Curry, Matthew J. and Terrel, Andy R. and Roučka, Štěpán and Saboo, Ashutosh and Fernando, Isuru and Kulal, Sumith and Cimrman, Robert and Scopatz, Anthony. SymPy: symbolic computing in Python. *PeerJ Computer Science*, 3:e103, 2017.

[4] Ronald L. Rivest, Adi Shamir, and Leonard Adleman. A method for obtaining digital signatures and public-key cryptosystems. *Communications of the ACM*, 21(2):120–126, 1978.

[5] Game Freak and Creatures. *Pokémon Red Version*. Nintendo, Game Boy, 1996.

[6] Root Beer. Straightedge and Compass – Multiplication is Constructible. YouTube video. https://www.youtube.com/watch?v=XeroUXDPsdA. Accessed March 16, 2026.

[7] Root Beer. Straightedge and Compass – Squareroot is Constructible. YouTube video. https://www.youtube.com/watch?v=mfGquoIA3k4. Accessed March 16, 2026.

[8] Omer Goldzweig. CasNum repository. https://github.com/0x0mer/CasNum